

Lab 5: Basic SQL

Use lab5 folder

- Start mysql> at **lab5** directory
- Save the file and extract it under Lab folder
- Remember to start by creating a silicon database to use.

As root user

```
mysql> create database silicon;  
mysql> grant all privileges on silicon.* to 'jelee';
```

As development user (e.g., 'jelee' here)

```
mysql> use silicon;
```

A. Data Definition Language (DDL)

A1. Create table

% Create EMPLOYEE table (To save time, COPY and PASTE the script from cr_emp.sql)

```
mysql> CREATE TABLE employee  
-> (fname          VARCHAR(8),  
->  minit          VARCHAR(2),  
->  lname          VARCHAR(8),  
->  ssn            VARCHAR(9) NOT NULL,  
->  bdate          DATE,  
->  address        VARCHAR(27),  
->  sex            VARCHAR(1),  
->  salary         INT(7) NOT NULL,  
->  superssn       VARCHAR(9),  
->  dno            INT(1) NOT NULL) ;
```

Query OK, 0 rows affected (0.18 sec)

% Describe Table

```
mysql> desc employee;
```

Field	Type	Null	Key	Default	Extra
fname	varchar(8)	YES		NULL	
minit	varchar(2)	YES		NULL	
lname	varchar(8)	YES		NULL	
ssn	varchar(9)	NO		NULL	
bdate	date	YES		NULL	
address	varchar(27)	YES		NULL	
sex	varchar(1)	YES		NULL	
salary	int(7)	NO		NULL	
super_ssn	varchar(9)	YES		NULL	
dno	int(1)	NO		NULL	

% run script on mysql client

Use arrow to see the last command
mysql>

Execute SQL from a file (e.g., in another cmd.exe (Win) to create cr_emp.sql file using notepad.exe or any text editor. And copy and paste CREATE TABLE employee . . . Then, save it. For Mac users, vim on terminal or textedit can be used to create cr_emp.sql file.

```
mysql> source cr_emp.sql;
ERROR 1050 (42S01): Table 'employee' already exists
```

You will get an error, since name is already used by an existing table

```
mysql> drop table employee;
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> commit;
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> source cr_emp.sql;
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> describe employee;
```

Field	Type	Null	Key	Default	Extra
fname	varchar(8)	YES		NULL	
minit	varchar(2)	YES		NULL	
lname	varchar(8)	YES		NULL	
ssn	varchar(9)	NO		NULL	
bdate	date	YES		NULL	
address	varchar(27)	YES		NULL	
sex	varchar(1)	YES		NULL	
salary	int(7)	NO		NULL	
superssn	varchar(9)	YES		NULL	
dno	int(1)	NO		NULL	

```
10 rows in set (0.03 sec)
```

% DROP Table

```
mysql> DROP TABLE employee;
```

% Create Table EMPLOYEE with Primary Key

```
mysql> CREATE TABLE employee
-> (fname          VARCHAR(8),
->  minit          VARCHAR(2),
->  lname          VARCHAR(8),
->  ssn            VARCHAR(9) NOT NULL,
->  bdate          DATE,
->  address         VARCHAR(27),
->  sex            VARCHAR(1),
->  salary          INT(7) NOT NULL,
->  superssn       VARCHAR(9),
```

```
-> dno          INT(1) NOT NULL,
-> PRIMARY KEY (ssn) );
```

% Otherwise, you can modify cr_emp.sql using editor, then source cr_emp.sql;

% Create Table DEPARTMENT with PRIMARY Key (To save time, COPY and PASTE the script from cr_company.sql and add PRIMARY KEY)

```
mysql> DROP TABLE department;
mysql> CREATE TABLE department(dnumber      INT(1),
->  dname          VARCHAR(15),
->  mgr_ssn        VARCHAR(9),
->  mgr_start_date DATE,
->  PRIMARY KEY (dnumber) );
```

% Create Table EMPLOYEE with Foreign Key (To save time, COPY and PASTE the script from cr_company.sql and add PRIMARY KEY and FOREIGN KEY)

```
mysql> DROP TABLE employee;

mysql> CREATE TABLE employee (fname          VARCHAR(8),
->  minit          VARCHAR(2),
->  lname          VARCHAR(8),
->  ssn            VARCHAR(9) NOT NULL,
->  bdate          DATE,
->  address        VARCHAR(27),
->  sex            VARCHAR(1),
->  salary          INT(7) NOT NULL,
->  super_ssn      VARCHAR(9),
->  dno            INT(1) NOT NULL,
->  PRIMARY KEY (ssn),
->  FOREIGN KEY (dno) REFERENCES department (dnumber)
->  ON DELETE CASCADE );
```

```
mysql> insert into employee values
-> ('Joe', 'M', 'Smith', '123456789', NOW(),
-> '123 Smith St.', 'm', 45000, '123456789', 1);
```

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key
constraint fails (`silicon`.`employee`, CONSTRAINT `fk_dno` FOREIGN KEY
(`dno`) REFERENCES `department` (`dnumber`) ON DELETE CASCADE)
```

% Why? And how to resolve the issue?

```
mysql> insert into department values (1,'HEADQUARTERS','888665555',NOW());
```

```
mysql> insert into employee values
-> ('Joe', 'M', 'Smith', '123456789', NOW(),
-> '123 Smith St.', 'm', 45000, '123456789', 1);
```

Query OK, 1 row affected, 1 warning (0.05 sec)

```
mysql> select * from employee;
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+
```

```

| fname | minit | lname | ssn      | bdate      | address      | sex |
salary | super_ssn | dno |
+-----+-----+-----+-----+-----+-----+-----+
| Joe   | M     | Smith | 123456789 | 2017-09-27 | 123 Smith St. | m   |
45000 | 123456789 | 1 |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.08 sec)

```

A2. Alter Table

```

mysql> describe department;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| dnumber        | int(1)        | NO   | PRI | NULL    |       |
| dname          | varchar(15)   | YES  |     | NULL    |       |
| mgr_ssn        | varchar(9)    | YES  |     | NULL    |       |
| mgr_start_date | date          | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.03 sec)

```

% Modify Table

```

mysql> ALTER TABLE department
-> ADD (manager VARCHAR(8)) ;
Query OK, 0 rows affected (0.08 sec)
Records: 0 Duplicates: 0 Warnings: 0

```

```

mysql> DESC department;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| dnumber        | int(1)        | NO   | PRI | NULL    |       |
| dname          | varchar(15)   | YES  |     | NULL    |       |
| mgr_ssn        | varchar(9)    | YES  |     | NULL    |       |
| mgr_start_date | date          | YES  |     | NULL    |       |
| manager        | varchar(8)    | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.03 sec)

```

% What is difference?

```

mysql> ALTER TABLE department
-> MODIFY COLUMN manager VARCHAR(15);

```

```

mysql> DESC department;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| dnumber        | int(1)        | NO   | PRI | NULL    |       |
| dname          | varchar(15)   | YES  |     | NULL    |       |
| mgr_ssn        | varchar(9)    | YES  |     | NULL    |       |
| mgr_start_date | date          | YES  |     | NULL    |       |
| manager        | varchar(15)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+

```

```
5 rows in set (0.03 sec)
% What is difference?
```

A3. Create and Drop Index

```
mysql> CREATE INDEX employee_ssn_idx
      -> ON employee (ssn);
Query OK, 0 rows affected (0.08 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DROP INDEX employee_superssn_idx
      -> ON employee;
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

A4. Create and Drop View

% Create View emp_dno_1 limited in columns (fname, lname, dno) and set of data (WHERE dno=1) from the employee table.

```
mysql> CREATE VIEW emp_dno_1
      -> AS SELECT fname, lname, dno
      -> FROM employee
      -> WHERE dno = 1;
Query OK, 0 rows affected (0.08 sec)

mysql> show full tables;

mysql> desc emp_dno_1;

mysql> select * from emp_dno_1;

mysql> DROP VIEW emp_dno_1;
```

% Create View dept_manager using join

```
mysql> CREATE VIEW dept_managers AS
      -> SELECT dnumber, dname, mgr_ssn, lname, fname
      -> FROM    employee, department
      -> WHERE   employee.ssn = department.mgr_ssn ;
Query OK, 0 rows affected (0.05 sec)

mysql> desc dept_managers;
```

% Create View dept_average_salary using aggregate function.

```
mysql> CREATE VIEW dept_average_salary AS
      -> SELECT dnumber, dname, AVG(salary) AS average_salary
      -> FROM    department, employee
      -> WHERE   employee.dno = department.dnumber
      -> GROUP BY dnumber, dname ;
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> desc dept_average_salary;
```

B. Data Manipulation Language (DML)

B0. Create COMPANY Database

```
mysql> source cr_company.sql
```

% If you want to create COMPANY database with Primary Key and Foreign Keys, use the following script:

```
mysql> source cr_company_full.sql
```

```
+-----+
| Tables_in_lab |
+-----+
| department    |
| dependent     |
| dept_average_salary |
| dept_locations |
| dept_managers |
| emp_department_1 |
| employee      |
| project       |
| works_on      |
+-----+
9 rows in set (0.05 sec)
```

B1. SELECT

% Every query in B.1, B.2 and B3 is scripted on the file named 'Q#.sql' in which # indicates the query number. For example, to run 'Query 0', type

```
mysql> source Q0.sql
```

% **SELECT - FROM - WHERE**

Query 0: Retrieve the birth date and address of the employee(s) whose name is 'Joe M. SMITH'.

```
mysql> SELECT bdate, address
-> FROM employee
-> WHERE fname = 'JOHN'
-> AND minit = 'B'
-> AND lname = 'SMITH';
```

Or

```
mysql> source Q0.sql
```

```
+-----+-----+
| bdate      | address |
+-----+-----+
```

```
| 2022-05-08 | 731 FONDREN, HOUSTON, TX |
+-----+
1 row in set (0.03 sec)
```

Query 1: Retrieve the name and address of all employees who work for the 'RESEARCH' department.

```
mysql> SELECT fname, lname, address
-> FROM employee, department
-> WHERE dname = 'RESEARCH'
-> AND dnumber = dno;
+-----+-----+-----+
| fname | lname | address |
+-----+-----+-----+
| JOHN  | SMITH | 731 FONDREN, HOUSTON, TX |
| FRANKLIN | WONG  | 638 VOSS,HOUSTON TX |
| RAMESH | NARAYAN | 975 FIRE OAK, HUMBLE, TX |
| JOYCE  | ENGLISH | 5631 RICE, HOUSTON, TX |
+-----+-----+-----+
4 rows in set (0.05 sec)
```

Query 2: For every project located in 'STAFFORD', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

```
mysql> SELECT pnumber, dnum, lname, address, bdate
-> FROM project, department, employee
-> WHERE dnum=dnumber
-> AND mgr_ssn = ssn
-> AND plocation = 'STAFFORD';
+-----+-----+-----+-----+-----+
| pnumber | dnum | lname | address | bdate |
+-----+-----+-----+-----+-----+
| 10 | 4 | WALLACE | 291 BERRY, BELLAIRE, TX | 2017-08-21 |
| 30 | 4 | WALLACE | 291 BERRY, BELLAIRE, TX | 2017-08-21 |
+-----+-----+-----+-----+-----+
2 rows in set (0.06 sec)
```

% Aliasing

Query 1A:

```
mysql> SELECT fname, employee.lname, address
-> FROM employee, department
-> WHERE department.dname = 'RESEARCH'
-> AND department.dnumber = employee.dno;
+-----+-----+-----+
| fname | lname | address |
+-----+-----+-----+
| JOHN  | SMITH | 731 FONDREN, HOUSTON, TX |
| FRANKLIN | WONG  | 638 VOSS,HOUSTON TX |
| RAMESH | NARAYAN | 975 FIRE OAK, HUMBLE, TX |
| JOYCE  | ENGLISH | 5631 RICE, HOUSTON, TX |
+-----+-----+-----+
4 rows in set (0.05 sec)
```

Query 8: For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

```
mysql> SELECT E.fname, E.lname, S.fname, S.lname
-> FROM employee E, employee S
-> WHERE E.super_ssn = S.ssn;
```

fname	lname	fname	lname
JOHN	SMITH	FRANKLIN	WONG
RAMESH	NARAYAN	FRANKLIN	WONG
JOYCE	ENGLISH	FRANKLIN	WONG
ALICIA	ZELAYA	JENNIFER	WALLACE
AHMAD	JABBAR	JENNIFER	WALLACE
FRANKLIN	WONG	JAMES	BORG
JENNIFER	WALLACE	JAMES	BORG

7 rows in set (0.06 sec)

% missing WHERE

Queries 9 and 10: Select all EMPLOYEE ssns (Q9) and all combinations of EMPLOYEE ssn and DEPARTMENT dname (Q10) in the database.

```
mysql> SELECT ssn
-> FROM employee;

mysql> SELECT ssn, dname
-> FROM employee, department;
(How many rows are returned?)
```

% Asterisk

Q1c:

```
mysql> SELECT *
-> FROM employee
-> WHERE DNO = 5;
```

% Set operations in SQL (DISTINCT, UNION)

Query 11: Retrieve the salary of every employee and all distinct salary values.

```
mysql> SELECT ALL salary
-> FROM employee;

mysql> SELECT DISTINCT salary
-> FROM employee;
```

% What is the difference?

Query 4: Make a list of all project numbers for projects that involve an employee whose last name is 'SMITH', either as a worker or as a manager of the department that controls the project.

```
mysql> SELECT DISTINCT pnumber
-> FROM project, department, employee
-> WHERE dnum = dnumber
-> AND mgr_ssn = ssn
-> AND lname = 'SMITH'
```



```

-> UNION
-> SELECT DISTINCT pnumber
-> FROM project, works_on, employee
-> WHERE pnumber = pno
-> AND essn = ssn
-> AND lname = 'SMITH';

```

% Pattern matching (LIKE)

Query 12: Retrieve all employees whose address is in Houston, Texas.

```

mysql> SELECT fname, lname
-> FROM employee
-> WHERE address LIKE '%HOUSTON, TX%';

```

Query 12A: Find all employees who were born during the 2020s.

```

mysql> SELECT fname, lname
-> FROM employee
-> WHERE bdate LIKE '__2____';
(2 times of '_' + '1' + 7 times of "_")

```

Query 13: Show the resulting salaries if every employee working on the 'PRODUCTX' project is given a 10 percent raise.

```

mysql> SELECT fname, lname, 1.1*salary AS INCREASED_SAL
-> FROM employee, works_on, project
-> WHERE ssn=essn AND pno=pnumber AND pname='PRODUCTX';

```

Query 14: Retrieve all employees in department 5 whose salary is between \$30,000 and \$40,000.

```

mysql> SELECT *
-> FROM employee
-> WHERE (salary BETWEEN 30000 AND 40000) AND dno = 5;

```

Query 15: Retrieve a list of employees and the projects they are working on, ordered by department and, within each department, ordered alphabetically by last name, first name.

```

mysql> SELECT dname, lname, fname, pname
-> FROM department, employee, works_on, project
-> WHERE dnumber=dno AND ssn=essn AND pno=pnumber
-> ORDER BY dname, lname, fname;

```

B2. Complex SELECT

% NULL

Query 18: Retrieve the names of all employees who do not have supervisors.

```

mysql> SELECT fname, lname
-> FROM employee
-> WHERE super_ssn IS null;

```

#replace with NOT NULL

```
mysql> SELECT fname, lname
->      FROM employee
->      WHERE super_ssn IS NOT null;
```

```
+-----+-----+
| fname | lname |
+-----+-----+
| JOHN  | SMITH  |
| FRANKLIN | WONG   |
| ALICIA | ZELAYA |
| JENNIFER | WALLACE |
| RAMESH | NARAYAN |
| JOYCE  | ENGLISH |
| AHMAD  | JABBAR  |
+-----+-----+
```

7 rows in set (0.05 sec)

% Nested Queries

Q4A:

```
mysql> SELECT DISTINCT pnumber
->      FROM project
->      WHERE pnumber IN (SELECT pnumber
->                        FROM project, department, employee
->                        WHERE dnum=dnumber
->                        AND mgr_ssn=ssn
->                        AND lname = 'SMITH')
->      OR pnumber IN (SELECT pno
->                     FROM works_on, employee
->                     WHERE essn=ssn
->                     AND lname = 'SMITH');
```

Query 16: Retrieve the name of each employee who has a dependent with the same first name and same sex as the employee.

```
mysql> SELECT E.fname, E.lname
->      FROM employee E
->      WHERE E.ssn IN (SELECT essn
->                     FROM dependent
->                     WHERE E.fname = dependent_name
->                     AND E.sex=sex);
```

% Correlated Nested queries

Q16A:

```
mysql> SELECT E.fname, E.lname
->      FROM employee E, dependent D
->      WHERE E.ssn=D.essn
->      AND E.sex=D.sex
->      AND E.fname=D.dependent_name;
```

Query 3: Retrieve the name of each employee who works on all the projects controlled by department number 4.

```
mysql> SELECT fname, lname
->      FROM employee
```

```

-> WHERE NOT EXISTS (SELECT pnumber FROM project
->                     WHERE dnum=4 AND pnumber NOT IN
->                     (SELECT pno FROM works_on WHERE employee.ssn=works_on.essn));

```

Q3B:

```

mysql> SELECT      fname, lname
->   FROM      employee
->   WHERE      NOT EXISTS (SELECT *
->                           FROM works_on B
->                           WHERE (B.pno IN (SELECT pnumber
->                                           FROM      project
->                                           WHERE      dnum=4)
->
->                           AND
->                           NOT EXISTS (SELECT *
->                                           FROM      works_on C
->                                           WHERE      C.essn=ssn
->                                           AND      C.pno=B.pno)))));

```

% Explicit Sets

Query 17: Retrieve the social security numbers of all employees who work on project number 1,2, or 3.

```

mysql> SELECT  DISTINCT essn
->   FROM      works_on
->   WHERE      pno IN (1,2,3);

```

% Outer Joins

Q8C:

```

mysql> SELECT E.lname, S.lname
->   FROM      employee E left outer join employee S
->   ON      E.super_ssn = S.ssn;

```

% Aggregate Functions

Query 19: Find the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary.

```

mysql> SELECT  SUM(salary), MAX(salary), MIN(salary), AVG(salary)
->   FROM      employee;

```

Query 20: Find the sum of the salaries of employees in the 'RESEARCH' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

```

mysql> SELECT  SUM(salary), MAX(salary), MIN(salary), AVG(salary)
->   FROM      employee, department
->   WHERE      dno=dnumber AND dname='RESEARCH';

```

Queries 21 and 22: Retrieve the total number of employees in the company (Q21) and the number of employees in the 'RESEARCH' department (Q22).

```

mysql> SELECT      COUNT(*)
->   FROM      employee;

```

```

mysql> SELECT  COUNT(*)

```

```
-> FROM employee, department
-> WHERE dno=dnumber AND dname='RESEARCH';
```

Query 23: Count the number of distinct salary values in the database.

```
mysql> SELECT COUNT(DISTINCT salary) FROM employee;
```

% Grouping (GROUP BY, HAVING)

Query 24: For each department, retrieve the department number, the number of employees in the department, and their average salary.

```
mysql> SELECT dno, COUNT(*), AVG(salary)
-> FROM employee
-> GROUP BY dno;
```

Query 25: For each project, retrieve the project number, the project name, and the number of employees who work on that project.

```
mysql> SELECT pnumber, pname, COUNT(*)
-> FROM project, works_on
-> WHERE pnumber=pno
-> GROUP BY pnumber, pname;
```

Query 26: For each project, on which more than two employees work, retrieve the project name, and the number of employees who work on the project.

```
mysql> SELECT pnumber, pname, COUNT(*)
-> FROM project, works_on
-> WHERE pnumber=pno
-> GROUP BY pnumber, pname
-> HAVING COUNT(*) > 2;
```

Query 27: For each project, retrieve the project number, the project name, and the number of employees from department 5 who work on the project.

```
mysql> SELECT pnumber, pname, COUNT(*)
-> FROM project, works_on, employee
-> WHERE pnumber=pno AND ssn=essn AND dno=5
-> GROUP BY pnumber, pname;
```

Query 28: For each department that has more than three employees, retrieve the department number and the number of its employees who are making more than \$25,000.

```
mysql> SELECT dnumber, COUNT(*)
-> FROM department, employee
-> WHERE dnumber=dno AND salary>25000
-> AND dno in (SELECT dno
-> FROM employee
-> GROUP BY dno
-> HAVING COUNT(*)>3)
-> GROUP BY dnumber;
```

B3. Insert, Delete, and Update

% INSERT

U1:

```
mysql> INSERT INTO employee
-> VALUES ('RICHARD','K','MARINI','653298653','1969-04-30',
->          '98 OAK FOREST, KATY, TX','M',37000,'653298653',4)
```

```
mysql> INSERT INTO employee (fname, lname, dno, ssn, salary)
-> VALUES ('RICHARD','MARINI',4,'653298653', 40000);
```

U2:

```
mysql> INSERT INTO employee (fname, lname, ssn, dno,salary)
-> VALUES ('ROBERT','HATCHER','980735245',2,40000);
```

```
mysql> INSERT INTO employee (fname, lname, ssn, dno)
-> VALUES ('ROBERT','HATCHER','980735245',2);
(Why error?)
```

U3:

```
mysql> CREATE TABLE depts_info
-> (dept_name varchar(15),
-> no_of_emps int(2),
-> total_sal int(6));
```

```
mysql> INSERT INTO depts_info (dept_name, no_of_emps, total_sal)
-> SELECT dname, count(*), sum(salary)
-> FROM department, employee
-> WHERE dnumber = dno
-> GROUP BY dname;
```

```
mysql> select * from depts_info;
```

% DELETE

U4:

```
mysql> DELETE FROM employee WHERE lname = 'BROWN';
mysql> DELETE FROM employee WHERE ssn='123456789';
mysql> DELETE FROM employee
-> WHERE dno in (SELECT dnumber
->                FROM department
->                WHERE dname='RESEARCH');
```

% UPDATE

U5:

```
mysql> UPDATE project
-> SET plocation='BELLAIRES', dnum=5
-> WHERE pnumber = 10;
```

U6:

```
mysql> UPDATE employee
-> SET salary=salary*1.1
-> WHERE dno IN (SELECT dnumber
->                FROM department
->                WHERE dname = 'RESEARCH');
```

Exercise

Exercise 1: Creating and Altering Table

As an exercise, create a table called **students** with the following columns and data types:

Column Name	Data Type
StudentID	INT(5) NOT NULL
Name	VARCHAR(25)
Major	VARCHAR(15)
GPA	DECIMAL(6,3)

Create another table called **courses** with the following columns and data types:

Column Name	Data Type
StudentID	INT(5) NOT NULL
CourseNumber	VARCHAR(15) NOT NULL
CourseName	VARCHAR(25)
Semester	VARCHAR(10)
Year	INT(4)
Grade	VARCHAR(2)

Use the DESCRIBE command to display the data types of the columns after each table is created.

Next, use the ALTER TABLE statement to add the following column to the **students** table:

Column Name	Data Type
TutorID	INT(5)

Use the ALTER TABLE statement to define the StudentID as the PRIMARY KEY for the **students** table.

Use the ALTER TABLE statement to define the StudentID and CourseNumber as the PRIMARY KEY for the **courses** table. To do this, list both of the column names separated by a comma.

Use the ALTER TABLE statement to define StudentID in the **courses** table as a FOREIGN KEY that references the StudentID in the **students** table.

Finally, add some data to the **students** and **courses** tables (simply copy and paste these statements into your script file to add the data):

```
INSERT INTO students VALUES (101, 'Bill', 'CIS', 3.45, 102);
INSERT INTO students VALUES (102, 'Mary', 'CIS', 3.10, NULL);
INSERT INTO students VALUES (103, 'Sue', 'Marketing', 2.95, 102);
INSERT INTO students VALUES (104, 'Tom', 'Finance', 3.5, 106);
INSERT INTO students VALUES (105, 'Alex', 'CIS', 2.75, 106);
INSERT INTO students VALUES (106, 'Sam', 'Marketing', 3.25, 103);
INSERT INTO students VALUES (107, 'Jane', 'Finance', 2.90, 102);
INSERT INTO courses VALUES (101, 'CIS3400', 'DBMS I', 'FALL', 1997, 'B+');
INSERT INTO courses VALUES (101, 'CIS3100', 'OOP I', 'SPRING', 1999, 'A-');
INSERT INTO courses VALUES (101, 'MKT3000', 'Marketing', 'FALL', 1997, 'A');
INSERT INTO courses VALUES (102, 'CIS3400', 'DBMS I', 'SPRING', 1997, 'A-');
INSERT INTO courses VALUES (102, 'CIS3500', 'Network I', 'SUMMER', 1997, 'B');
INSERT INTO courses VALUES (102, 'CIS4500', 'Network II', 'FALL', 1997, 'B+');
INSERT INTO courses VALUES (103, 'MKT3100', 'Advertise', 'SPRING', 1998, 'A');
INSERT INTO courses VALUES (103, 'MKT3000', 'Marketing', 'FALL', 1997, 'A');
```

```
INSERT INTO courses VALUES (103, 'MKT4100', 'Marketing II', 'SUMMER', 1998, 'A-');
```

Exercise 2: More Queries with UNIVERSITY database

Using UNIVERSITY database that you created in Exercise 1, create SQL below:

1. Average GPA of all students
2. Average GPA of Finance and CIS students
3. Give the name of the student with the highest GPA
4. Show the students with the GPA grades in each major
5. Show the students with the GPA grades in each major ordered by GPA in descending order
6. Provide a listing of each student and the name of their tutor (Recursive query). Note that recursive query is only available on MySQL 8.X. So, if you are using 5.X, you can ignore this query.
7. How many students does each tutor work with?
8. Add .05 to all of the Marketing major's GPA's
9. Change Sam's tutor from Sue to Jane
10. For any student who is currently majoring in CIS and who has a GPA of less than 3.0, change their major to Marketing.